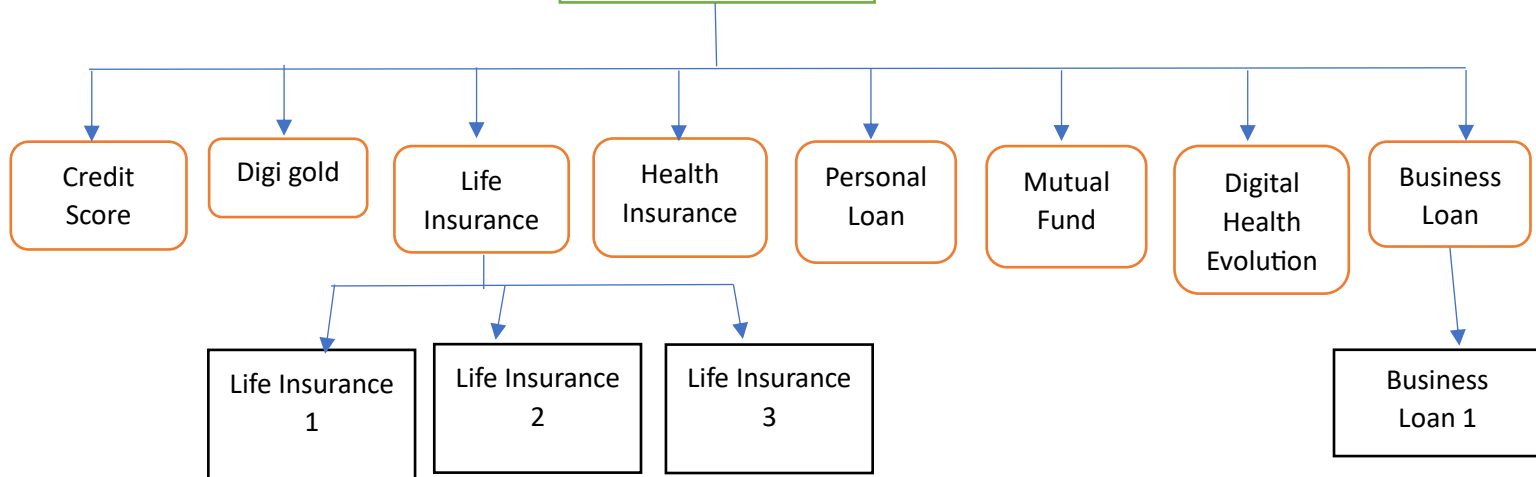
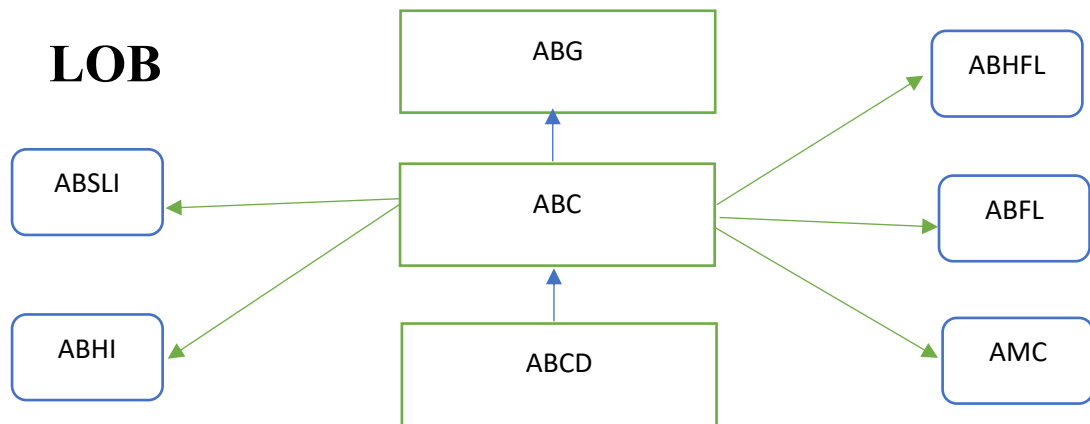


PROJECT ARCHITECTURE OVERFLOW

LOB



Abbreviations

ABG: Aditya Birla Group

ABC: Aditya Birla Capital

ABCD: Aditya Birla Capital Digital

Environment

1. Dev: Development Env
2. SIT: System Integrated Testing
3. UAT: User Acceptance Testing
4. CUG/PROD: Pre-Production Environment
5. Super PROD: Production Environment



My SQL Scripts – App DB

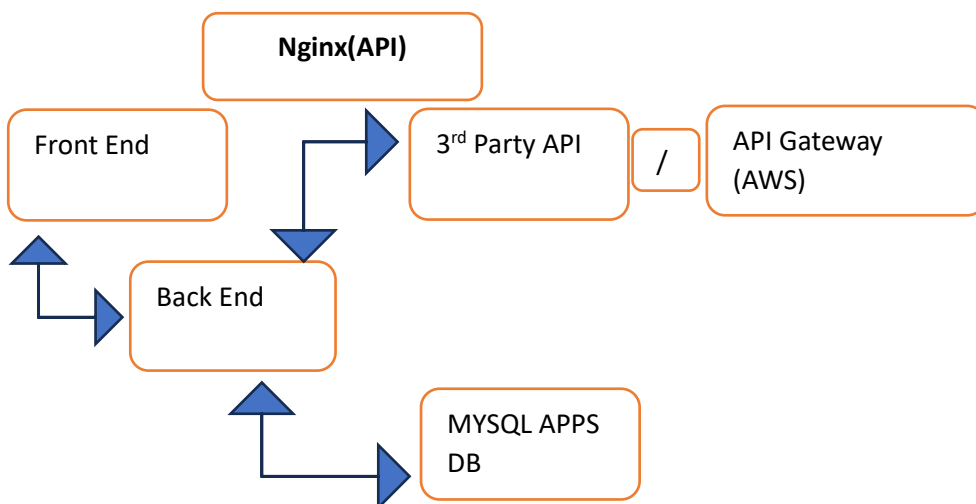
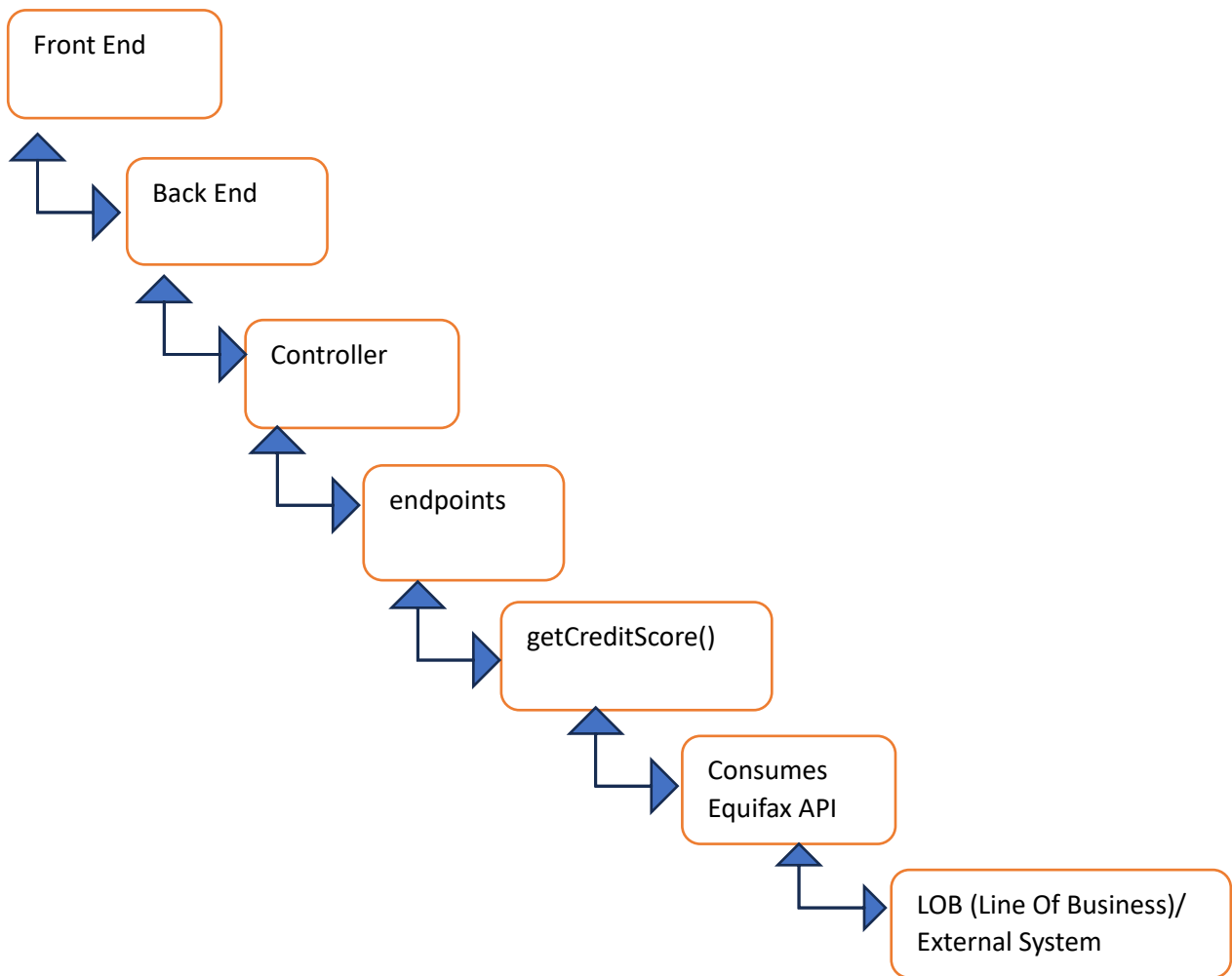
1. Dev: Development Env
2. SIT: System Integrated Testing
3. UAT: User Acceptance Testing
4. CUG/PROD: Pre-Production Environment
5. Super PROD: Production Environment



Pre-requisites

1. Java-17 version
2. Eclipse/STS/IntelliJ IDEA
3. MySQL
4. Notepad++
5. Postman
6. Google Cloud SDK
7. GitHub

Project Execution Flow



Project Setup Steps

Step 1: Install Git Desktop on your system and sign in using your Corporate ID.

Step 2: Clone Digigold source code using Git Desktop.

Step 3: Open the Digigold codebase in Spring Tool Suite.

Step 4: Configure the Lombok dependency in Spring Tool Suite by placing the Lombok JAR file inside the STS installation directory.

Step 5: Create a development profile by adding an **application-dev.properties** file within the Digigold project. Ensure that this profile is activated by setting **spring.profiles.active=dev** in the **application.properties** file.

Step 6: Update the sql port number in **application-dev.properties**,

Change the port number in **spring.datasource.url** from 3306 to 3307.

Note: For every Microservices, We can find our environment **application.properties** file in Git repository searching by environment name(Like Dev, UAT, Prod). It will be like **dev-env-tracking-devops**.

When all the configurations are done and try to start , if we get the 8080 port is already being used , then kill the port and restart the application .

Commands to find and kill the 8080 PID:

Cmd-1: netstat -ano | findstr :8080

Cmd-2: taskkill /PID 8416 /F

DB Configuration Steps

Step-1: Create a new connection by clicking the  icon.

Step-2: Enter the following details for the new connection.

Username: abcone_db

Password: ABC@123 (select test connection to enter the password)

Port: 3307

Branching Strategies

When starting development, follow these steps:

Step-1: Create a feature branch from dev.

Step-2: Commit your local changes to your feature branch.

Step-3: Update the version text file as needed.

Step-4: Raise a pull request (PR) to merge your feature branch into Dev.

Once the code moved to the Dev branch, it will be moved through the following stages which requires two level of approvals:

1. QA
2. SIT (System Integration Testing)
3. UAT (User Acceptance Testing)
4. MAIN

SIT, UAT – we have to raise a JSM ticket through Jira for approval by specifying the details of the scripts or changes required.

Two types of JSM tickets – Non Prod and Prod

✓ Non Prod = DB scripts

We have to give the rollback script while raising a JSM ticket like Delete the current updates.

- ✓ Prod = Actual Code Approval
Git Hub changes must be specified like “yes”.

-No need of Git hub changes when raising ticket for the DB Script changes.

-Ensure that all necessary information, such as the environment details and test scripts, are included.

Follow Up:

After the JSM request is created and processed, coordinate with your testing team to schedule and perform SIT and UAT.

Monitor the tests and address any issues that arise.

Note: Ensure that the version file is updated along with the changes.

To ensure version by using Git:

- To check in Dev

dev-env-tracking-devops/***branch*** -> helm_version.txt

- To check in SIT

sit-env-tracking-devops/***branch*** -> helm_version.txt

- To check in UAT

uat-env-tracking-devops/***branch*** -> helm_version.txt

- To check in Prod

prod-env-tracking-devops/***branch*** -> helm_version.txt

- To check in SuperProd

superprod-env-tracking-devops/***branch*** -> helm_version.txt

Even in prod if some hot fix is given that also maintained here in the repo.

Ex: 1.0.1-hotfix-1

Hotfix(requires one week) Merging:

When a hotfix is merged directly into the main branch, you should also merge this hotfix into the dev and QA branches to ensure consistency across all environments.

Merging to dev and QA:

After the hotfix is merged into main, create pull requests to merge the hotfix into the dev and QA branches.

Ensure that you test these merges to avoid introducing any issues.